System and Devices (SYS3)

Lecture Review – Week 8

Dr Pengcheng Liu

Welcome

Check-In Code:

633209

Log into checkin.york.ac.uk, find the session and enter the code.

Valid until until Thursday, 16 April 2026 11:00

TEF Gold Teaching Excellence Framework

Week 8 Module
Overview and Updates

Module Overview and Updates

- **Week 8**:

Lecture: In Week 8, our discussion was focused on Virtual Memory and Page Replacement in Lecture 13 and Lecture 14, respectively. And we also had the 3rd tutorial lecture on Static & Dynamic Scheduling.

Lab: Register renaming. In previous weeks, we explored register hazards and we noted that creating delays in the pipline to solve these problems is not very efficient, as it wastes CPU clock cycles. An alternative method, register renaming, attempts to solve some of these hazards by another method, which avoids inserting stall cycles. This lab sesison explores that idea and a solution.

Exercises/Quizzes: Exercise 3 and Quiz 3 will be released on 17/04/2026 (Week 8 Friday). Quiz 3 will be discussed in the Lecture Review session on 23/04/2026 (Week 9 Thursday). Exercise 3 solution will be released on 24/04/2026 (Week 9 Friday) . Exercise 3 will be discussed in the Lecture Review session on 30/04/2026 (Week 10 Thursday).

Module Overview and Updates

- **Next Week (Week 9)**:

Lecture: In Week 9, our discussion will be focused on some principles of cache memory, and block replacement & write strategy in Lecture 15 and Lecture 16, respectively.

Lab: Instruction Reordering. Instruction reordering can be used for a number of reasons, to **reduce hazards**, including structural hazards, and data hazards. In this exercise we focus on using reordering mainly as a way to improve performance where register hazards persist. For example, we know from previous exercises that RAW hazards cannot be eliminated by renaming, although it can eliminate WAR and WAW hazards. Reordering the instruction sequence can sometimes help to improve if not eliminate these problems. This session will investigate reordering and look at ideas for solving this problem.

Questions on Padlet

Padlet Questions

TL2 Pipeline Hazards – Q3: The video said that without operand forwarding the ID stage must happen after the WB stage of the previous instruction but as register read/writes can happen in half a cycle so you read in the first half and write in the second why can't they happen in the same cycle reducing the CPI to 3 instead of 4.

True or False: For a MIPS instruction SW R2, 16(R3), some contents stored in its ID/EX pipeline register will bypass the EX unit directly to EX/MEM pipeline register.

MEM: In this stage, the data from the source register R2 is prepared to be written to memory. The calculated memory address from the execution stage is passed on to the MEM stage, along with the data to be stored (R2's content).
WB: This stage is typically empty for store instructions since they don't read from memory.

The bypassing mechanism:

Padlet Questions

TL2 Pipeline Hazards – Q3: The video said that without operand forwarding the ID stage must happen after the WB stage of the previous instruction but as register read/writes can happen in half a cycle so you read in the first half and write in the second why can't they happen in the same cycle reducing the CPI to 3 instead of 4.

True or False: For a MIPS instruction SW R2, 16(R3), some contents stored in its ID/EX pipeline register will bypass the EX unit directly to EX/MEM pipeline register.

The bypassing mechanism:

In the MIPS architecture, bypassing is used to forward data directly from one pipeline stage to another to avoid pipeline stalls caused by data hazards. For store instructions like SW, there's a potential data hazard between the execution and memory access stages because the effective memory address calculated in the execution stage needs to be forwarded directly to the memory access stage (MEM).

Therefore, some contents stored in the ID/EX pipeline register (specifically, the calculated effective address) will indeed bypass the EX unit directly to the EX/MEM pipeline register for store instructions like SW in the MIPS architecture. This is necessary to ensure correct operation and to prevent pipeline stalls due to data hazards.

Padlet Questions

TL2 Pipeline Hazards – Q3: The video said that without operand forwarding the ID stage must happen after the WB stage of the previous instruction but as register read/writes can happen in half a cycle so you read in the first half and write in the second why can't they happen in the same cycle reducing the CPI to 3 instead of 4.

Without operand forwarding, the ID stage of an instruction must happen after the WB stage of the previous instruction to ensure that the data needed by the current instruction is available from the register file. This sequential flow is necessary to maintain correct program behavior and avoid hazards.

However, performing register reads and writes within the same cycle isn't feasible in practice in the context of a pipelined processor like MIPS, even if it seems like it could reduce the CPI or they can theoretically be done in half a cycle each.

In a pipelined processor, each stage operates in parallel, and instructions are processed concurrently through different stages of the pipeline. If register reads and writes were both attempted in the same cycle, it would essentially require merging the ID and WB stages into a single stage, violating the fundamental concept of pipelining, where each stage operates independently and in parallel.

Padlet Questions

TL2 Pipeline Hazards – Q3: The video said that without operand forwarding the ID stage must happen after the WB stage of the previous instruction but as register read/writes can happen in half a cycle so you read in the first half and write in the second why can't they happen in the same cycle reducing the CPI to 3 instead of 4.

Performing register writes (WB) in the same cycle as register reads (ID) could introduce data hazards. For example, if an instruction in the EX stage writes to a register and the subsequent instruction in the ID stage attempts to read from the same register in the same cycle, a hazard occurs because the data hasn't been written yet. This would require additional logic to handle such hazards, potentially increasing the complexity of the processor and negating the benefits of a simple pipeline design.

Even if register reads and writes could theoretically be performed within the same cycle, it would require extremely precise timing control at the level of individual clock cycles. Achieving such precise timing control across all stages of the pipeline, especially in modern processors operating at high frequencies, is highly challenging and may not be practical.

Instead, techniques like **operand forwarding** or **out-of-order execution** are typically used.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

In Tomasulo's Algorithm, reservation stations are a fundamental component used for **implementing out-of-order execution** in superscalar processors. They allow instructions to be dispatched, executed, and completed independently of program order, thus maximizing instruction-level parallelism and improving performance.

The structure of reservation stations in a processor implementing Tomasulo's Algorithm typically **includes several fields** designed to facilitate out-of-order execution and dynamic scheduling. In the context of Tomasulo's Algorithm, reservation stations **act as buffers** that hold instructions until their operands are available and the execution resources are free.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

1. Operation (Op):

This field **specifies the type of operation** the instruction will perform.
Examples include addition, subtraction, multiplication, division, etc.
This field helps **identify the functional unit needed** for the instruction's execution.

2, 3. Qj and Qk (Operand Status Fields):

These fields **represent the status of the source operands** required by the instruction.
Qj and Qk **hold tags that identify the reservation stations** producing the required operands. If the required operand is not available, the corresponding Qj or Qk field holds a reservation station tag that produces the operand.
If the operand is available, these fields hold a special value indicating readiness.
These fields help resolve data dependencies by tracking the producers of operands.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

4, 5. Vj and Vk (Operand Value Fields):

These fields **hold the values of the source operands**.
If the operand is available, its value is stored directly in Vj or Vk.
If the operand is not available, these fields remain empty or hold stale data.
Vj and Vk, along with Qj and Qk, help determine when an instruction can proceed to execution.

6. A (Address/Immediate Field):

This field **holds an address or an immediate value used by the instruction**, if applicable.
For memory operations (e.g., load and store instructions), A holds the memory address.
For certain arithmetic or logical operations, A may hold an immediate value used in the operation.
This field is not used in all types of instructions but is crucial for those involving memory access or immediate data.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

7. Busy Bit:

The Busy bit **indicates whether the reservation station is currently occupied by an instruction**.
If the Busy bit is set (1), the reservation station is in use and cannot accept new instructions.
If the Busy bit is clear (0), the reservation station is available to accept a new instruction.

These fields collectively enable **dynamic scheduling and out-of-order execution** by allowing instructions to be held, operands to be tracked, and dependencies to be resolved efficiently. By utilizing reservation stations with these fields, a processor implementing Tomasulo's Algorithm can achieve higher instruction-level parallelism and better resource utilization compared to traditional in-order execution pipelines.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

An example:

1. ADD R1, R2, R3 // R1 = R2 + R3
2. MUL R4, R1, R5 // R4 = R1 * R5
3. SUB R6, R4, R7 // R6 = R4 - R7

Step 1: Dispatch Stage:
Instruction 1 (ADD R1, R2, R3) is dispatched to the Add RS.
Instruction 2 (MUL R4, R1, R5) is dispatched to the Mult RS.

Step 2: Operand Availability:
For instruction 1: The reservation station Add RS waits until both R2 and R3 are available.
For instruction 2: The reservation station Mult RS waits until both R1 and R5 are available.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

An example:

1. ADD R1, R2, R3 // R1 = R2 + R3
2. MUL R4, R1, R5 // R4 = R1 * R5
3. SUB R6, R4, R7 // R6 = R4 - R7

Step 3: Issue Stage:
Once all operands are available and the execution units are free, instructions are issued for execution.
Let's assume that R2, R3, and R5 are available, and the Add and Mult functional units are free.
Instruction 1 is issued for execution, performing the addition operation.
Instruction 2 is issued for execution, performing the multiplication operation.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

An example:

1. ADD R1, R2, R3 // R1 = R2 + R3
2. MUL R4, R1, R5 // R4 = R1 * R5
3. SUB R6, R4, R7 // R6 = R4 - R7

Step 4: Result Forwarding:
After the execution of each instruction completes, the result is broadcasted to all reservation stations.
Any subsequent instructions that depend on the result can then proceed.

Padlet Questions

S2 Week 6 L9 - Reservation Stations and Tomasulo's Algorithm: *Can you explain more about how reservation stations work and their application in Tomasulo's Algorithm? I'm struggling to understand exactly how a RS gets its inputs and the conditions it needs to then perform an operation.*

Conditions for Operation:

All **source operands must be available**, either from the register file or from previous instructions that produced them. The reservation station waits until all operands are ready before proceeding.

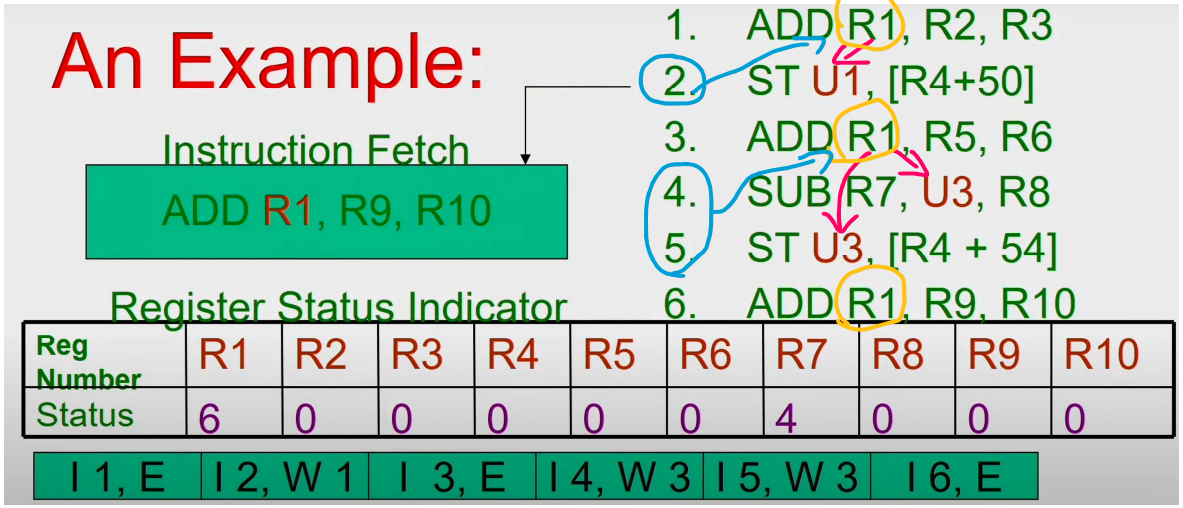
The associated **execution unit (e.g., adder, multiplier) must be free** to perform the operation. If the unit is busy, the reservation station waits until it becomes available.

The reservation station checks for any resource conflicts or structural hazards that might prevent the operation from proceeding. For example, ensuring that there are **no data hazards or conflicts with other instructions** using the same functional unit.

Padlet Questions

S2 Week 6 L10 - Tomasulo's Algorithm Example Query: *In this example, when you do register renaming, is line 4 correct? Should U3 not be R1 as you haven't stored the result of line 3 in U3 yet? If it is correct, where have I misunderstood?*

For the 1st instruction, since R2 and R3 are available, the result of R1 will be produced by unit number 1. For the 3rd instruction, since R5 and R6 are available, the result of R1 will be produced by unit number 3, and act as an operand for 4h and 5th instructions. So operand forwarding has happened automatically, from R1 (in 1st instruction) to U1, and from R1 (in 3rd instruction) to U3 (in 4th and 5th instructions). Register renaming is also happened here as different functional units are producing different versions of R1 at different times, and we are going to write to R1 at different times as well. 2nd Instruction wants the results of R1 from 1st instruction, while 4th and 5th instructions want the results of R1 from 3rd instruction.



Padlet Questions

S2 Week 7 L11 - Speculative Execution/Tomasulo's Algorithm with 5 stage pipeline: *Could you please explain how the speculative execution/tomasulo's algorithm combines with the 5 stage pipeline model we have previously been discussing? I understand them in isolation, but I don't understand how they would work together.*

Speculative execution and Tomasulo's algorithm are advanced techniques used to improve the performance of processors by executing instructions **out of order and concurrently**, while the MIPS 5-stage pipeline model is a classic approach to organizing and executing instructions in **a sequential manner**.

The classic 5-stage pipeline model consists of the following stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).
Each stage of the pipeline is responsible for executing a specific part of the instruction, allowing multiple instructions to be processed simultaneously.

Padlet Questions

S2 Week 7 L11 - Speculative Execution/Tomasulo's Algorithm with 5 stage pipeline: *Could you please explain how the speculative execution/tomasulo's algorithm combines with the 5 stage pipeline model we have previously been discussing? I understand them in isolation, but I don't understand how they would work together.*

Speculative execution is a technique where the processor **predicts the outcome of a branch instruction** and begins **executing the predicted instructions before the branch is resolved**.

In the RISC pipeline, speculative execution could occur during the Instruction Fetch (IF) stage, where the processor predicts the outcome of branch instructions and fetches and decodes subsequent instructions based on the prediction.

Padlet Questions

S2 Week 7 L11 - Speculative Execution/Tomasulo's Algorithm with 5 stage pipeline: *Could you please explain how the speculative execution/tomasulo's algorithm combines with the 5 stage pipeline model we have previously been discussing? I understand them in isolation, but I don't understand how they would work together.*

Tomasulo's algorithm is a dynamic scheduling and out-of-order execution technique used to improve instruction-level parallelism and resource utilization. It utilizes **reservation stations** and **a reorder buffer** to manage dependencies and execute instructions out of order.

In the RISC pipeline, Tomasulo's algorithm could be applied during the Execute (EX) stage, where instructions are executed out-of-order based on the availability of functional units and operands.

Padlet Questions

S2 Week 7 L11 - Speculative Execution/Tomasulo's Algorithm with 5 stage pipeline: *Could you please explain how the speculative execution/tomasulo's algorithm combines with the 5 stage pipeline model we have previously been discussing? I understand them in isolation, but I don't understand how they would work together.*

An example of combining speculative execution and Tomasulo's algorithm.

In the RISC pipeline, speculative execution could be used to **fetch and decode instructions ahead of branch decisions**, while Tomasulo's algorithm could be applied to **execute instructions out-of-order** and exploit available functional units efficiently.

For example, if a branch instruction is encountered in the IF stage, speculative execution could predict the branch outcome and fetch subsequent instructions accordingly. Meanwhile, Tomasulo's algorithm could allow other instructions to proceed through the pipeline and execute out-of-order, maximizing resource utilization.

Padlet Questions

S2 Week 7 L11 - Speculative Execution/Tomasulo's Algorithm with 5 stage pipeline: *Could you please explain how the speculative execution/tomasulo's algorithm combines with the 5 stage pipeline model we have previously been discussing? I understand them in isolation, but I don't understand how they would work together.*

If the branch prediction is correct, the speculatively executed instructions **continue processing as normal**. If the prediction is incorrect, the speculatively executed instructions are **flushed from the pipeline**, and correct instructions are fetched and executed based on the actual branch outcome.

By combining speculative execution with Tomasulo's algorithm in the RISC pipeline, processors can achieve higher performance by overlapping instruction execution and minimizing pipeline stalls. This results in improved instruction throughput and overall system efficiency.

Padlet Questions

L8 Compiler Techniques Slide 13- Strip Mining:

In Strip mining, what are the pair of loops? If we have an unknown number of loop iterations, where does the pair come from, are the loops in that pair different?

Strip Mining

Unknown number of loop iterations?

- Goal: make k copies of the loop body (Number of iterations = n)
- Generate pair of loops:
  - First executes n mod k times
  - Second executes n/k tims
  - Strip mining
- Example: Let n=35, k=4
  - Loop 1 executes 3 times (35%4 = 3)
  - Loop 2 executes 8 times by unrolling 4 copies per iteration

Strip mining is usually used in compilers for vector processors. In real programs we do not usually know the upper bound on the loop. Suppose it is n, and we would like to unroll the loop to make k copies of the body. Instead of a single unrolled loop, we generate a pair of consecutive loops. The first executes **(n mod k) times** and has a body that is the original loop. The second is the unrolled body surrounded by an outer loop that iterates (n/k) times.

Padlet Questions

S2 Week 7 L12 - Superscalar processors Slide 15: *How is DADDIU executing at clock cycle 5 but SD executes at clock cycle 3. How would it know to store the correct information as #1 wont of been added to R2 yet.*

Example (Without Speculation)

Iteration number	Instructions	Issues at clock cycle number	Executes at clock cycle number	Memory access at clock cycle number	Write CDB at clock cycle number	Comment
1	LD R2,0(R1)	1	2	3	4	First issue
1	DADDIU R2,R2,#1	1	5		6	Wait for LW
1	SD R2,0(R1)	2	3	7		Wait for DADDIU
1	DADDIU R1,R1,#8	2	3		4	Execute directly
1	BNE R2,R3,LOOP	3	7			Wait for DADDIU

In this example, we assume a case of dual issue, i.e., issue width=2, at every cycle we can issue 2 instructions. DADDIU can be issued at clock cycle number 1, but it cannot be executed as it needs R2 which is available only at the clock cycle number 4. So DADDIU executes at clock cycle number 5. As it is a dual issue, SD can be issued at clock cycle number 2, when it is issued at clock cycle number 2, it can be executed at 3 for the calculation of the effective address.

Padlet Questions

S2 Week 7 L12 - Superscalar processors Slide 15: *How is DADDIU executing at clock cycle 5 but SD executes at clock cycle 3. How would it know to store the correct information as #1 wont of been added to R2 yet.*

Example (Without Speculation)

Iteration number	Instructions	Issues at clock cycle number	Executes at clock cycle number	Memory access at clock cycle number	Write CDB at clock cycle number	Comment
1	LD R2,0(R1)	1	2	3	4	First issue
1	DADDIU R2,R2,#1	1	5		6	Wait for LW
1	SD R2,0(R1)	2	3	7		Wait for DADDIU
1	DADDIU R1,R1,#8	2	3		4	Execute directly
1	BNE R2,R3,LOOP	3	7			Wait for DADDIU

In a pipelined processor, instructions are fetched and executed in sequence, but the effects of an instruction may not be visible immediately. In this case, while DADDIU executes at clock cycle 5, its result isn't written back to register R2 until clock cycle 6. However, the SD instruction doesn't require the updated value of R2; it simply writes the current content of R2 to memory. Since the value in R2 is already available (the original value before the addition), it can be written to memory without waiting for the result of DADDIU to be written back. Therefore, SD can execute and complete its memory access stage at clock cycle 3, even though the result of DADDIU hasn't been applied yet.

Padlet Questions

S2 Week 7 L11 - Speculative execution - Operations with ROB - Commit:

Could you explain what happens in the Commit stage of ROB in greater detail? I'm struggling to understand the example provided in the lecture.

The Commit stage of the ROB is where the speculative execution decisions are finalized, and instructions are either committed to the architectural state or invalidated **based on the correctness of speculation**, ensuring precise and accurate program execution.

A bit more details:

Commit: This is the final stage of an instruction's execution, where its result is deemed final and is committed to the architectural state of the processor. After this stage, only the instruction's result remains visible to subsequent instructions.
Different sequences of actions at commit: (1) **Branch with an incorrect prediction**: If a branch with an incorrect prediction reaches the head of the ROB, it signifies that the speculation based on this branch was incorrect. (2) **Store**: When a store instruction reaches the head of the ROB, the memory is updated with the value specified by the instruction. (3) **Other instruction** (normal commit): For normal instructions, the processor updates the Register File (RF) with the result and removes the instruction from the ROB.

Padlet Questions

S2 Week 7 L11 - Speculative execution - Operations with ROB - Commit:

Could you explain what happens in the Commit stage of ROB in greater detail? I'm struggling to understand the example provided in the lecture.

Normal commit: In the case of a normal instruction, when it reaches the head of the ROB and its result is present in the buffer, the processor updates the RF with the result and removes it from the ROB. This ensures that the instruction's result is committed to the architectural state of the processor.

Committing a store: Similar to normal commit, but for store instructions, the memory is updated with the value specified by the store instruction instead of updating a result register.

Incorrect prediction of a branch: If a branch with an incorrect prediction reaches the head of the ROB, it indicates that the speculation based on this branch was wrong. This necessitates corrective action.

Flushing the ROB: In case of an incorrect branch prediction, the ROB is flushed. This means that all speculative instructions following the mispredicted branch are invalidated, and execution is restarted at the correct successor of the branch.

Correctly predicted branch: If the branch was correctly predicted, the branch is considered finished, and execution continues as expected without any need for corrective action.

Padlet Questions

S2 Week 7 L11 - Speculative execution - Operations with ROB - Issue: *For the 3rd bullet point in the slide shown, where is the slot number (that's taken from the ROB) for that instruction stored?*

During the issue stage, an instruction is taken from the instruction queue and is issued if there is an empty reservation station and an empty slot in the ROB. Since every instruction has a position in the ROB until it commits, we take a slot or tag a result using the ROB entry number rather than using the reservation station number.

Operations with ROB – Issue

- Get an instruction from the instruction queue.
- Issue the instruction if there is an empty reservation station and an empty slot in the ROB.
- Get a slot number (tag) from ROB for this instruction.
- Send the operands to the RS if they are available in either RF or in the ROB. (**Speculative register read**)
- The number of the ROB entry is also sent to the RS to tag the result when it is placed on the CDB.
- If either all reservations are full or the ROB is full, then instruction issue is stalled until both have available entries.

Padlet Questions

S2 Week 7 L11 - Speculative execution - Operations with ROB - Issue: *For the 3rd bullet point in the slide shown, where is the slot number (that's taken from the ROB) for that instruction stored?*

This tagging requires that the ROB assigned for an instruction must be tracked in the reservation station. The number of the ROB entry (i.e., ROB slot number) allocated for the result is tagged in the reservation station, so that the number can be used to tag the result when it is placed on the CDB.

Operations with ROB – Issue

- Get an instruction from the instruction queue.
- Issue the instruction if there is an empty reservation station and an empty slot in the ROB.
- Get a slot number (tag) from ROB for this instruction.
- Send the operands to the RS if they are available in either RF or in the ROB. (**Speculative register read**)
- The number of the ROB entry is also sent to the RS to tag the result when it is placed on the CDB.
- If either all reservations are full or the ROB is full, then instruction issue is stalled until both have available entries.

Padlet Questions

S2 Week 7 L12 - Superscalar processors – VLIW processors: *Would you be able to explain why there is only 1 DADDUI instruction in the table please? I didn't quite understand it from the lecture.*

In this example of DADDUI instruction, we are unrolling the operation of R1=R1-8, we only need to do it once, i.e., R1=R1-56, as all other pointing to R1 are automatically taken care of by the loop unrolling. We have discussed loop unrolling with another similar example in Lecture 8.

VLIW Processors

- Example VLIW processor:
  - One integer instruction (or branch)
  - Two independent floating-point operations
  - Two independent memory references

			Clock cycle issued
Loop:	L.D	F0, 0(R1)	1
	stall		2
	ADD.D	F4, F0, F2	3
	stall		4
	stall		5
	S.D	F4, 0 (R1)	6
	DADDUI	R1, R1, #-8	7
	stall		8
	BNE	R1, R2, Loop	9

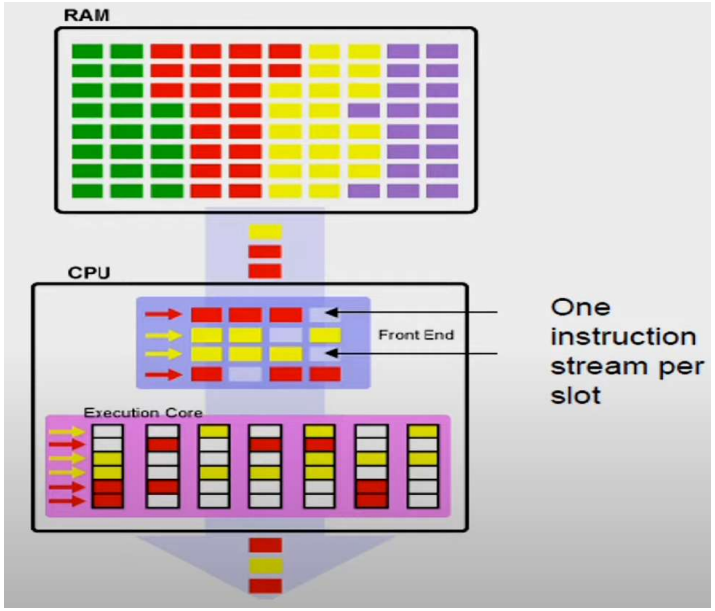
	Memory reference 1	Memory reference 2	FP operation 1	FP operation 2	Integer operation/branch
1	L.D F0,0(R1)	L.D F6,-8(R1)			
2	L.D F10,-16(R1)	L.D F14,-24(R1)			
3	L.D F18,-32(R1)	L.D F22,-40(R1)	ADD.D F4,F0,F2	ADD.D F8,F6,F2	
4	L.D F26,-48(R1)		ADD.D F12,F10,F2	ADD.D F16,F14,F2	
			ADD.D F20,F18,F2	ADD.D F24,F22,F2	
5	S.D F4,0(R1)	S.D F8,-8(R1)	ADD.D F28,F26,F2		
.	S.D F12,-16(R1)	S.D F16,-24(R1)			DADDUI R1,R1,#-56
.	S.D F20,24(R1)	S.D F24,16(R1)			
.	S.D F28,8(R1)				BNE R1,R2,Loop

VLIW instructions that occupy the inner loop and replace the unrolled sequence.

Performance: 9 cycles to complete 26 instructions. More registers used.

Padlet Questions

S2 Week 7 L12 - Superscalar processors – Multithreading: *I'm a bit confused by what multithreading is actually doing in this slide besides just allowing multiple instructions to use the same functional unit, could you explain please?*



You may have encountered the concept of multithreading in SYS2. Strictly speaking, multithreading uses thread-level parallelism, so we don't cover much on this topic. Multithreading **allows multiple threads to share the functional units** of a single processor in an overlapping fashion. Multithreading does not duplicate the entire processor as a multiprocessor does.

Instead, it shares most of the processor core among a set of threads, duplicating only private state, such as the registers and program counter. There are three main hardware approaches to multithreading**. Fine-grained multithreading** switches between threads on each clock, causing the execution of instructions from multiple threads to be interleaved. **Coarse-grained multithreading** switches threads only on costly stalls, such as level two or three cache misses. **Simultaneous multithreading** is a variation on fine- grained multithreading that arises naturally when fine-grained multithreading is implemented on top of a multiple-issue, dynamically scheduled processor.

Padlet Questions

S2 Week 8 TL3 - True/False 2: *Why is it false that it can't be issued if the operands aren't available as the ROB is not mentioned in the question and it is phrased around the availability of the operands. The question is asking if the availability of the operands stops the issuing not if anything else is needed.*

2. In a speculative dynamic scheduled processor, we can issue an instruction if there is an empty reservation station even though operands are not available.

Speculative dynamic scheduled processor always works with the help of a **reorder buffer** ROB. At the time of issue, we have to make sure that we get an entry in the ROB. So after fetching the instruction, before issuing into the reservation stations, before issuing into functional units, there are **2 conditions need to be satisfied**: if there is a **free reservation station entry** and if there is an entry allocated to this instruction in the **ROB**. Only if we have an entry in ROB and an entry in the reservation station, then only an instruction can be issued.

Padlet Questions

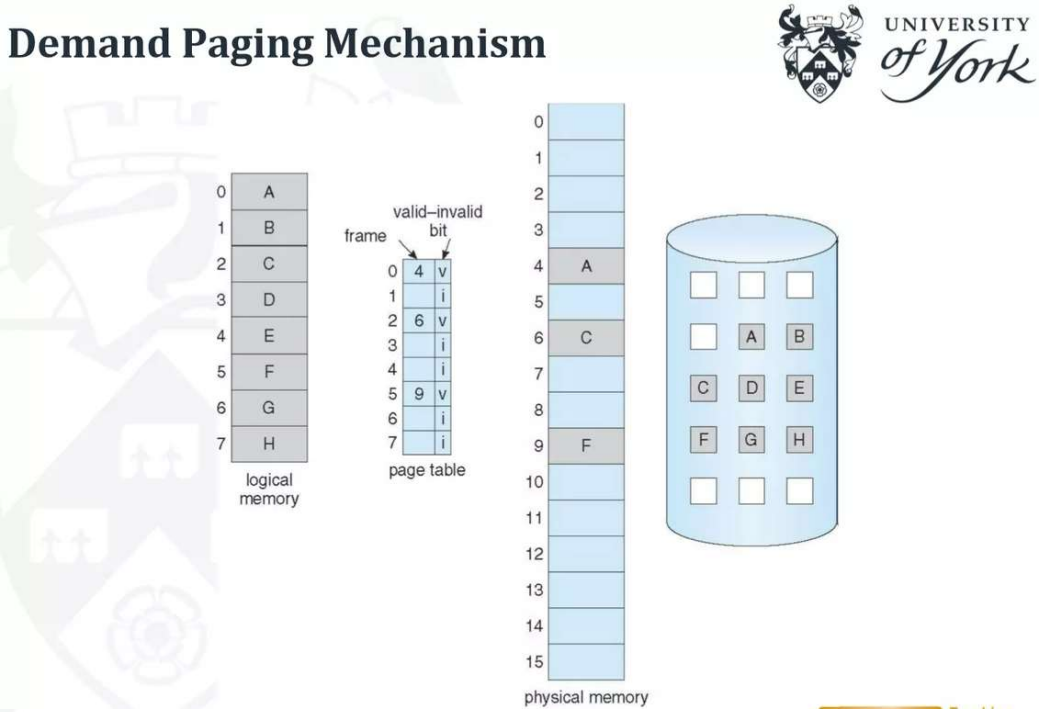
S2 Week 8 TL3 - True/False 2: *Why is it false that it can't be issued if the operands aren't available as the ROB is not mentioned in the question and it is phrased around the availability of the operands. The question is asking if the availability of the operands stops the issuing not if anything else is needed.*

2. In a speculative dynamic scheduled processor, we can issue an instruction if there is an empty reservation station even though operands are not available.

So in this question, the operands are available or not available is not the key point, as we can wait in the reservation station and move forward when previous instructions complete.
The key point is the two requirements in ROB and reservation station need to be met. We need to reserve a space in the ROB, when the ROB is full, we cannot issue, so issue is dependent on availability of space into the ROB and availability of entry in the reservation station.

Padlet Questions

S2 Week 8 L13 - Virtual memory Slide 13 Demand Paging Diagram: *I've got a bit of a misunderstanding with this diagram. I understand there's a page table, physical memory, logical memory, but what is the rightmost part? Does that just show logical memory as it would be in secondary storage?*



The rightmost part represents a **backing store**. When a page fault occurs, you will read the desired page from the backing store and store it in an available page frame in physical memory, and update the page table.

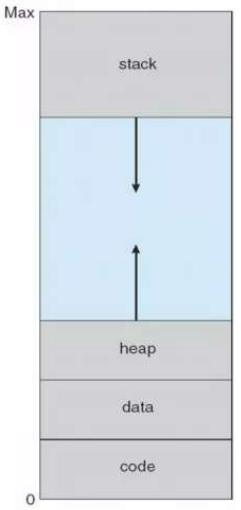
Padlet Questions

S2 Week 8 L13 - Virtual memory Slide 9 virtual address space: *Regarding the virtual address space, what indicates what goes in the heap and what goes in the stack?*

The stack is typically used to store local variables, procedure arguments, function parameters, and function return addresses. When a function is called, space is allocated on the stack for its local variables and parameters. When the function returns, the space is freed. The stack grows downwards in memory.

Virtual Address Space

- Usually design logical address space for stack to start at Max logical address and grow "down" while heap grows "up"
  - Maximizes address space use
  - Unused address space between the two
    - no physical memory needed until heap or stack grows to a given new page
- Enables sparse address spaces with holes left for growth, dynamically linked libraries, etc.
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during fork(), speeding process creation

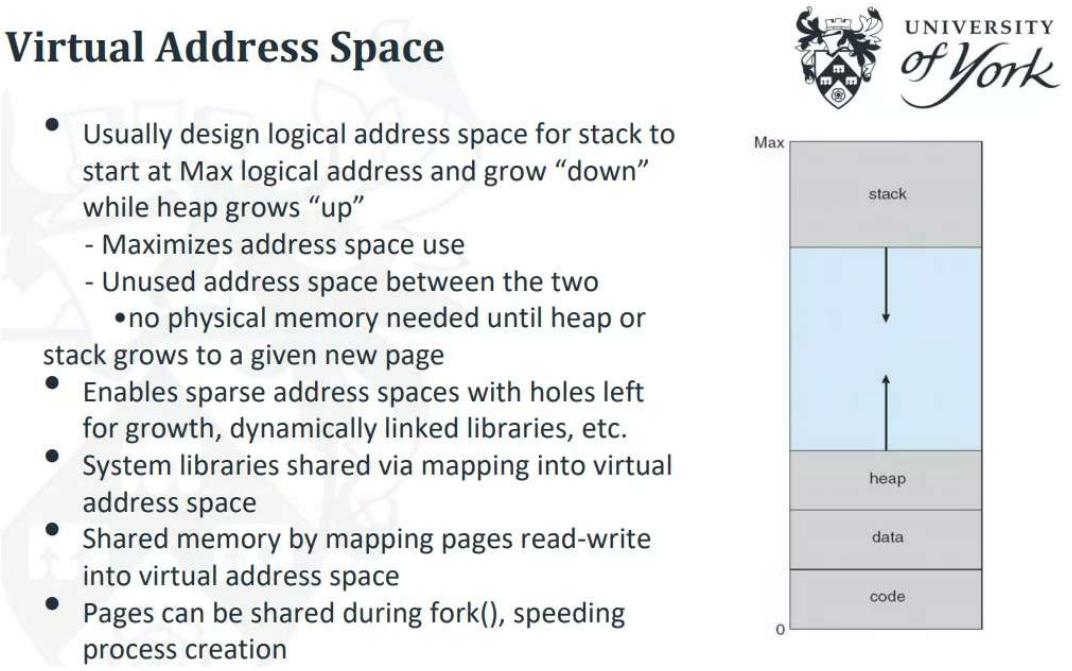


The heap contains the **dynamically allocated variables** to be used by the process. It is used for dynamic memory allocation, where memory is requested and released as needed during runtime.

Padlet Questions

S2 Week 8 L13 - Virtual memory Slide 9 virtual address space: *Regarding the virtual address space, what indicates what goes in the heap and what goes in the stack?*

The heap is a region of memory that is not managed automatically, so the programmer is responsible for explicitly requesting and releasing memory. Memory allocated on the heap must be released by the programmer, or it will result in a memory leak.



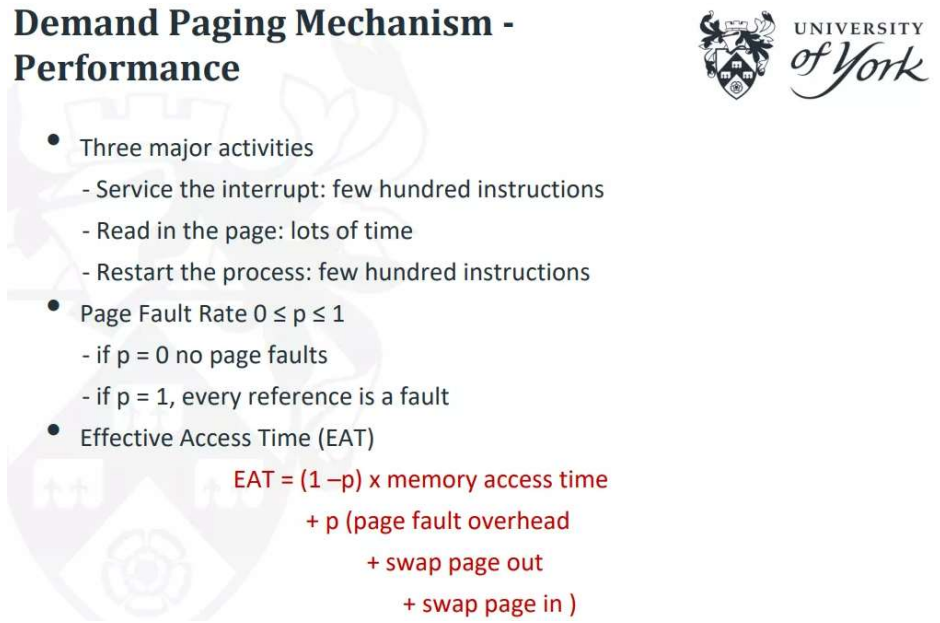
The heap grows upwards in memory. The compiler and OS make decisions on what goes in the heap and what goes in the stack based on the type and scope of the variables. Local variables and function parameters are typically allocated on the stack, while dynamically allocated variables are allocated on the heap.

Padlet Questions

S2 Week 8 L13 - Virtual memory Slide 18 demand paging mechanism:

When calculating EAT, I didn't understand why with memory access time you multiply by (1 - p) whereas with the other factors, p. Do you mind describing why this is the case?

In this equation, we let p be the probability of a page fault ($0 \leq p \leq 1$), i.e., p = probability of a page fault occurring, (1-p) = the probability of accessing memory in an available frame.



The bracket of items multiplied with p represents **the page fault time**, which is the sum of the additional overhead associated with accessing a page in the backing store. This includes additional context switches, disk latency and transfer time associated with page-in and page-out operations, the overhead of executing an operating system trap, etc.

SYS3 Exercises/Quizzes Discussions

Exercise 2

Exercise 2

Q1. Explain what is Delayed Branching?

Delayed branching is a technique used in computer processors to mitigate the performance impact of branching instructions, particularly conditional branches. In traditional pipelined processors, when a conditional branch instruction is encountered, the processor may need to stall, or temporarily pause, the pipeline until the outcome of the branch is known. This waiting period occurs because the processor cannot determine which instruction to fetch next until it resolves the branch condition.

Delayed branching works by **allowing the pipeline to continue executing instructions following the branch instruction**, regardless of whether the branch is taken or not. Instead of immediately branching to the target instruction specified by the branch, the processor continues fetching and executing instructions from the predicted path. Meanwhile, the branch condition is evaluated in parallel, and the decision about whether the branch was taken or not is made later in the pipeline.

Exercise 2

Q1. Explain what is Delayed Branching?

If the branch is not taken (i.e., the prediction was correct), the instructions fetched and executed after the branch are kept. However, if the branch is taken and the prediction was incorrect, the instructions fetched and executed after the branch are discarded. In this case, a pipeline flush occurs, and the correct target of the branch is fetched instead. This flushing of the pipeline incurs a performance penalty, but it is typically less severe than the stall that would occur if the pipeline were to wait for the branch resolution before proceeding.

Delayed branching helps improve the overall performance of the processor by increasing the instruction throughput and reducing the impact of branch mispredictions. However, it requires additional hardware support for speculative execution, branch prediction, and pipeline flushing mechanisms. Additionally, the effectiveness of delayed branching depends on the accuracy of branch prediction algorithms and the frequency of branch instructions in the program code.

Exercise 2

Q2. True or False: In a pipelined processor, control hazard penalties can be eliminated by dynamic branch prediction.

False.

Dynamic branch prediction can help mitigate, but not entirely eliminate, control hazards in a pipelined processor.

Control hazards occur when the pipeline must stall due to the uncertainty of a branch instruction's outcome. Dynamic branch prediction mechanisms, such as branch target buffers (BTBs) or branch history tables (BHTs), attempt to predict the outcome of branch instructions before they are resolved and guide the fetching of subsequent instructions.

Exercise 2

Q2. True or False: In a pipelined processor, control hazard penalties can be eliminated by dynamic branch prediction.

While dynamic branch prediction can significantly reduce the penalties associated with control hazards by speculatively fetching and executing instructions along the predicted path, it cannot completely eliminate them. In cases where the prediction is incorrect, the processor must **discard the speculatively executed instructions and refill the pipeline** with the correct instructions from the target of the branch. This process incurs a performance penalty known as a branch misprediction penalty.

Therefore, while dynamic branch prediction can effectively reduce the impact of control hazards, it cannot entirely eliminate them due to the inherent uncertainty in predicting the outcome of branch instructions.

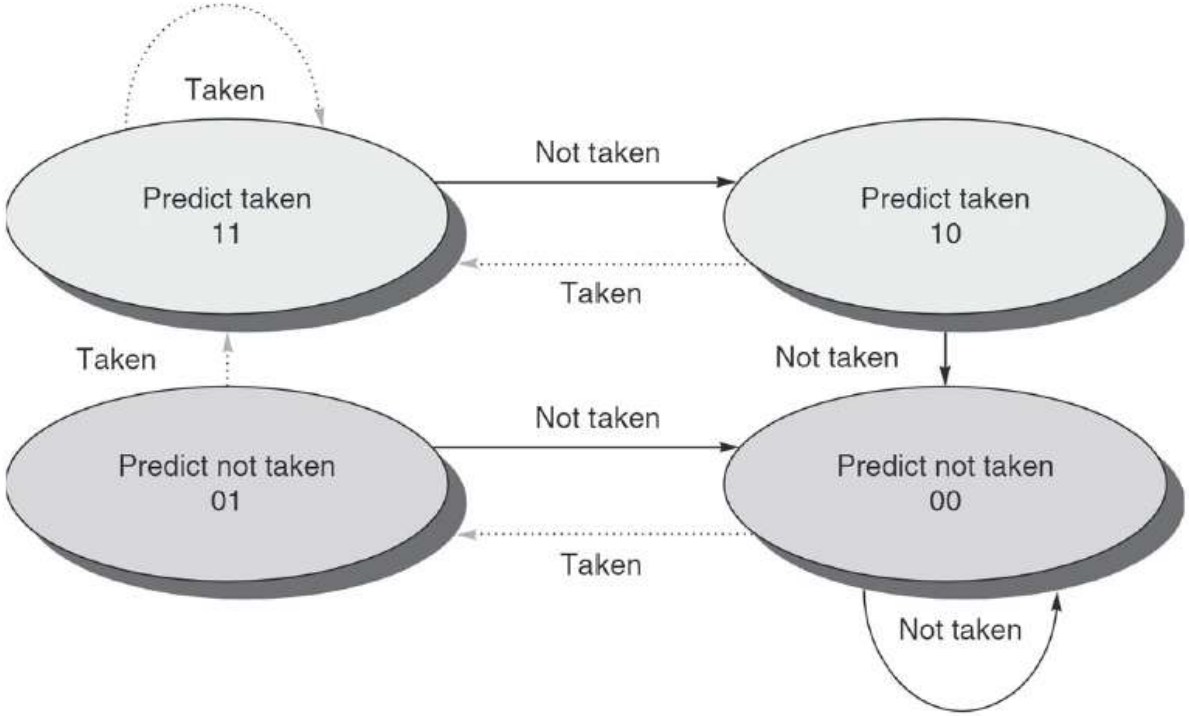
Exercise 2

Q3. We have a program core consisting of five conditional branches. The program core will be executed thousands of times. Below are the outcomes of each branch for one execution of the program core (T for taken, N for not taken).
Branch 1: T-T-T-T
Branch 2: N-N-N-N
Branch 3: N-T-N-T-N-T
Branch 4: T-N-T-N-T-N
Branch 5: T-T-N-T-T-N-T-T

Assume the behaviour of each branch remains the same for each program core execution. For the dynamic branch prediction schemes below, assume that each branch maps to a unique entry in the branch history table. Further assume that each BHT entry is initialized to the same stage before each execution

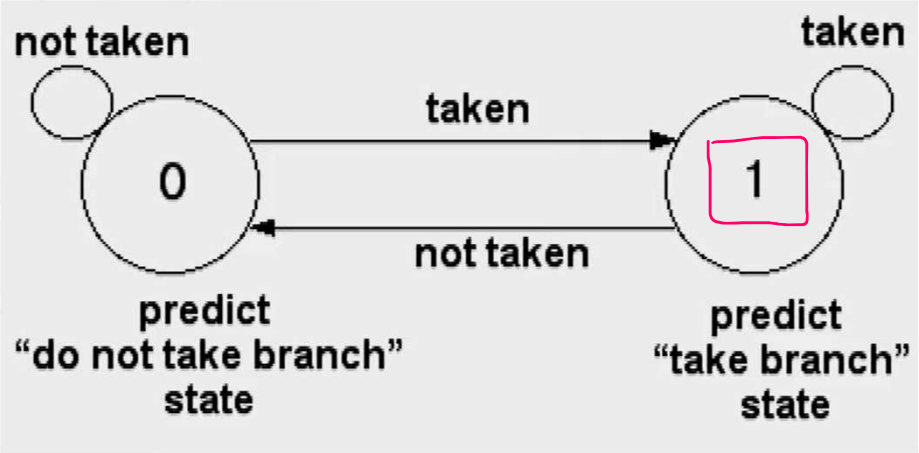
(a) Always taken
(b) Always not taken
(c) 1-bit predictor, initialized to predict taken
(d) 2-bit predictor (Finite State Machine as shown below), initialized to predict taken (10)

Exercise 2



What is the branch prediction accuracy for branch 1 to 5 with prediction scheme (a) through (d)?
Use the following table to fill out your final answer.

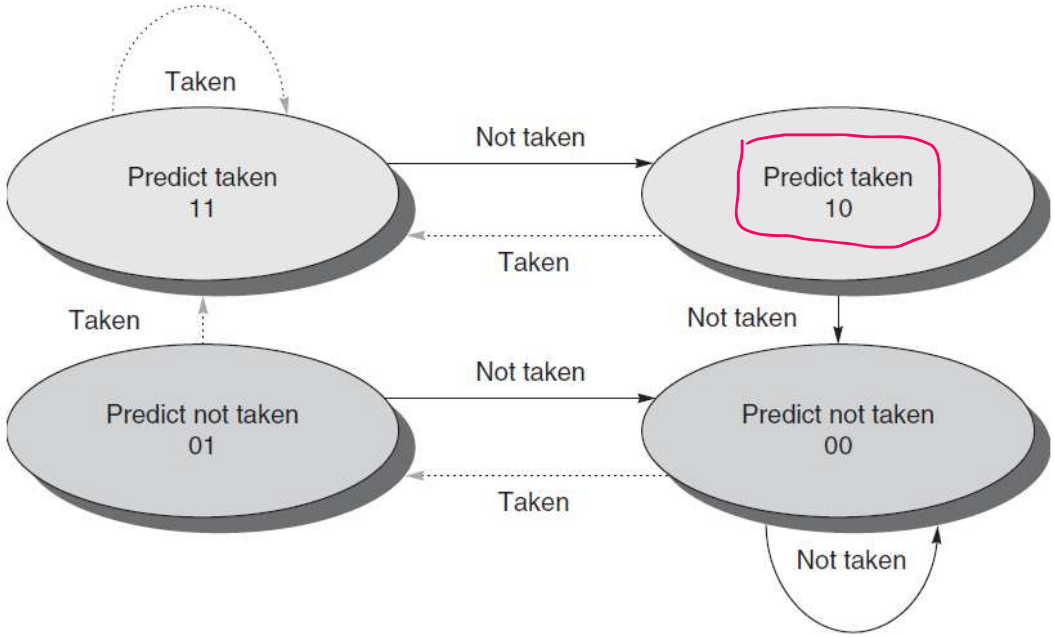
Exercise 2



Prediction results:

Branches	Prediction Schemes			
	(a)	(b)	(c)	(d)
1	T-T-T-T T-T-T-T	T-T-T-T N-N-N-N	T-T-T-T T-T-T-T	T-T-T-T T-T-T-T
2	N-N-N-N T-T-T-T	N-N-N-N N-N-N-N	N-N-N-N T-N-N-N	N-N-N-N T-N-N-N
3	N-T-N-T-N-T T-T-T-T-T-T	N-T-N-T-N-T N-N-N-N-N-N	N-T-N-T-N-T T-N-T-N-T-N	N-T-N-T-N-T T-N-N-N-N-N
4	T-N-T-N-T-N T-T-T-T-T-T	T-N-T-N-T-N N-N-N-N-N-N	T-N-T-N-T-N T-T-N-T-N-T	T-N-T-N-T-N T-T-T-T-T-T
5	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T	T-T-N-T-T-N-T-T N-N-N-N-N-N-N-N	T-T-N-T-T-N-T-T T-T-T-N-T-T-N-T	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T

Exercise 2



Prediction results:

Branches	Prediction Schemes			
	(a)	(b)	(c)	(d)
1	T-T-T-T T-T-T-T	T-T-T-T N-N-N-N	T-T-T-T T-T-T-T	T-T-T-T T-T-T-T
2	N-N-N-N T-T-T-T	N-N-N-N N-N-N-N	N-N-N-N T-N-N-N	N-N-N-N T-N-N-N
3	N-T-N-T-N-T T-T-T-T-T-T	N-T-N-T-N-T N-N-N-N-N-N	N-T-N-T-N-T T-N-T-N-T-N	N-T-N-T-N-T T-N-N-N-N-N
4	T-N-T-N-T-N T-T-T-T-T-T	T-N-T-N-T-N N-N-N-N-N-N	T-N-T-N-T-N T-T-N-T-N-T	T-N-T-N-T-N T-T-T-T-T-T
5	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T	T-T-N-T-T-N-T-T N-N-N-N-N-N-N-N	T-T-N-T-T-N-T-T T-T-T-N-T-T-N-T	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T

Exercise 2

Prediction results:

Branches	Prediction Schemes			
	(a)	(b)	(c)	(d)
1	T-T-T-T T-T-T-T	T-T-T-T N-N-N-N	T-T-T-T T-T-T-T	T-T-T-T T-T-T-T
2	N-N-N-N T-T-T-T	N-N-N-N N-N-N-N	N-N-N-N T-N-N-N	N-N-N-N T-N-N-N
3	N-T-N-T-N-T T-T-T-T-T-T	N-T-N-T-N-T N-N-N-N-N-N	N-T-N-T-N-T T-N-T-N-T-N	N-T-N-T-N-T T-N-N-N-N-N
4	T-N-T-N-T-N T-T-T-T-T-T	T-N-T-N-T-N N-N-N-N-N-N	T-N-T-N-T-N T-T-N-T-N-T	T-N-T-N-T-N T-T-T-T-T-T
5	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T	T-T-N-T-T-N-T-T N-N-N-N-N-N-N-N	T-T-N-T-T-N-T-T T-T-T-N-T-T-N-T	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T

Accuracies:

Branches	Prediction Schemes			
	(a)	(b)	(c)	(d)
1	100%	0%	100%	100%
2	0%	100%	75%	75%
3	50%	50%	0%	33.3%
4	50%	50%	16.7%	50%
5	75%	25%	50%	75%

Exercise 2

Q4. What is the shortcoming of Tomasulo's algorithm that leads to the introduction of speculation? How is the reorder buffer used to introduce speculation?

Tomasulo's algorithm is a dynamic scheduling algorithm designed to handle data hazards in pipelined processors by allowing instructions to execute out of order, based on the availability of operands. One of the key shortcomings of Tomasulo's algorithm is its inability to handle control hazards effectively.

Control hazards occur when the processor encounters branch instructions whose outcomes are uncertain. Since Tomasulo's algorithm primarily focuses on data hazards, it does not inherently address control hazards. As a result, when a branch instruction is encountered, the processor may need to stall the pipeline until the outcome of the branch is determined. This can lead to reduced instruction throughput and decreased processor efficiency.

Exercise 2

Q4. What is the shortcoming of Tomasulo's algorithm that leads to the introduction of speculation? How is the reorder buffer used to introduce speculation?

Speculation is introduced to address this limitation and improve performance. Speculative execution allows the processor to **predict the outcome of branch instructions** and continue executing instructions along the predicted path before the branch is resolved. If the prediction turns out to be correct, the speculatively executed instructions can be committed to the architectural state of the processor. However, if the prediction is incorrect, the speculatively executed instructions must be discarded, and the pipeline may need to be flushed to resume execution from the correct branch target.

The reorder buffer (ROB) is a key component used to introduce speculation in Tomasulo's algorithm. The ROB keeps track of the order of instructions as they are issued and completed in the pipeline. When a branch instruction is encountered, the processor can use the ROB to allow subsequent instructions to continue executing speculatively while the branch is being resolved. Speculatively executed instructions are kept in the ROB until their outcomes are confirmed. If the branch prediction is correct, the instructions are committed to the architectural state of the processor in program order. However, if the branch prediction is incorrect, the ROB can be used to discard the speculatively executed instructions and restore the pipeline to the correct execution path. This allows the processor to continue executing instructions efficiently while minimizing the performance impact of control hazards.

Exercise 2

Q5. (True or False) In superscalar mode, all the similar instructions are grouped and executed together.

In superscalar mode, all similar instructions **are not necessarily grouped and executed together**. Instead, the superscalar processor aims to exploit instruction-level parallelism by executing multiple instructions simultaneously, regardless of their similarity.

While superscalar processors do execute instructions in parallel, including similar instructions when possible, the primary focus is on maximizing instruction throughput by executing as many independent instructions as possible concurrently. This dynamic and flexible approach to instruction execution allows superscalar processors to achieve high performance across a wide range of workloads.

Instructions need to be independent of each other to be executed concurrently. Similar instructions may not always be independent. For example, if two consecutive instructions both modify the same register, they cannot be executed in parallel because the second instruction depends on the result of the first.

Exercise 2

Q5. (True or False) In superscalar mode, all the similar instructions are grouped and executed together.

Superscalar processors have a finite number of execution units. If all available execution units are occupied by instructions of the same type, other instructions, even if similar, have to wait for resources to become available.

Superscalar processors employ dynamic scheduling techniques to identify and execute independent instructions in parallel. This means the processor dynamically selects instructions from the instruction stream that are ready to execute, regardless of their type or similarity.

The goal of superscalar execution is to maximize instruction throughput by utilizing all available execution resources efficiently. This involves balancing the execution of different types of instructions to maintain high resource utilization.

Exercise 2

Q6. In super-scalar processors, out of order mode of execution is used, the results are stored in _____?

a) ROBs

b) Special memory locations

c) Temporary registers

d) TLB

In superscalar processors, the out-of-order execution mode allows instructions to be executed as soon as their operands are available, regardless of their original program order. This means that instructions can be completed and produce results out of program order.

To handle out-of-order execution and ensure correct program semantics, superscalar processors typically employ a structure known as **the reorder buffer (ROB)**. The ROB serves as a temporary storage mechanism for the results produced by instructions as they complete execution.

The ROB allows instructions to complete and produce results out of order, but it ensures that the results are committed to the architectural state of the processor in the correct program order. This means that instructions' results are only written back to the register file or memory when they reach the head of the ROB and all preceding instructions have committed their results.

Exercise 2

Q6. In super-scalar processors, out of order mode of execution is used, the results are stored in _____?

a) ROBs

b) Special memory locations

c) Temporary registers

d) TLB

The main purpose of using a reorder buffer (ROB) is to **facilitate out-of-order execution while preserving the correct program order**.

In out-of-order execution, instructions are executed as soon as their operands are available, regardless of their original program order. Storing results in the ROB allows the processor to preserve the correct program order. While instructions may complete execution out of order, the ROB ensures that the results are committed to the architectural state of the processor in the correct program order.

The ROB helps manage data dependencies between instructions. When an instruction completes execution, its result, along with its program sequence number (PSN), is stored in the ROB. This allows subsequent instructions that depend on the result of the completed instruction to proceed with execution once the operands become available.

Exercise 2

The ROB supports speculative execution by allowing instructions to execute ahead of branch resolution and other control hazards. Speculatively executed instructions are kept in the ROB until the correctness of their execution is confirmed. If a branch prediction is incorrect, the ROB can be used to discard the speculatively executed instructions and restore the pipeline to the correct execution path.

Storing results in the ROB ensures precise exception handling. Instructions' results are not immediately committed to the architectural state of the processor. Instead, they remain in the ROB until they reach the head of the buffer and all preceding instructions have been committed. This allows the processor to roll back or discard instructions in case of exceptions without affecting the architectural state.

In case of pipeline flushes due to mispredicted branches or other reasons, the ROB can be used to recover the correct execution state. Instructions that have not yet committed their results can be invalidated, and execution can be restarted from the correct program point.

Exercise 2

Q7. A special unit used to govern the out of order execution of the instructions is called as _____

a) Commitment unit

b) Temporal unit

c) Monitor

d) Supervisory unit

The commitment unit is primarily responsible for ensuring that the results of instructions are committed to the architectural state of the processor in the correct program order.

Once instructions have completed execution, their results are stored in a structure such as the reorder buffer (ROB). The commitment unit then ensures that the results are committed to the architectural state of the processor in the correct program order, based on the information stored in the ROB.

Exercise 2

Q8. The commitment unit uses a queue called _____

a) Reorder buffer

b) Commitment buffer

c) Storage buffer

d) None of the mentioned

The commitment unit typically uses a queue called the **"commit queue" or "commit buffer."**

The commit queue is a structure within the commitment unit that holds the results of instructions until they are ready to be committed to the architectural state of the processor. It ensures that instructions are committed in the correct program order, based on the information stored in the reorder buffer (ROB) or similar structure.

Exercise 2

Q8. The commitment unit uses a queue called _____

a) Reorder buffer

b) Commitment buffer

c) Storage buffer

d) None of the mentioned

As instructions complete execution and their results are stored in the ROB, they are queued in the commit queue. The commitment unit then processes instructions from the commit queue in program order, ensuring that the results are written back to the register file or memory in the correct sequence.

The commit queue helps manage the flow of completed instructions through the commitment unit, ensuring precise and orderly updates to the architectural state of the processor while allowing out-of-order execution to proceed efficiently.

Exercise 2

Q9. Consider the following instruction sequence:

1. LD R1, 0(R2)

2. ADD R3, R1, R4

3. SUB R5, R3, R6

4. MUL R7, R8, R9

5. ADD R10, R1, R11

Assume: Load latency = 2 cycles; ADD/SUB latency = 1 cycle; MUL latency = 3 cycles; In-order issue pipeline.

Answer the following questions:

a) Identify data dependencies

b) Perform static scheduling to minimize stalls

c) Draw the execution timeline

Solutions:

Identify dependencies

Dependencies:

2 depends on 1 (RAW on R1)

3 depends on 2 (RAW on R3)

5 depends on 1 (RAW on R1)

Instruction 4 is independent.

Exercise 2

Q9. Consider the following instruction sequence:

1. LD R1, 0(R2)
2. ADD R3, R1, R4
3. SUB R5, R3, R6
4. MUL R7, R8, R9
5. ADD R10, R1, R11

Assume: Load latency = 2 cycles; ADD/SUB latency = 1 cycle; MUL latency = 3 cycles; In-order issue pipeline.

Answer the following questions:

a) Identify data dependencies

b) Perform static scheduling to minimize stalls

c) Draw the execution timeline

Original execution (without scheduling)

Cycle	Instruction
1	LD
2	stall
3	ADD
4	SUB
5	MUL
6	ADD

Stalls occur because ADD waits for LD result.

Reorder independent instruction

Move instruction 4 earlier.

Optimized schedule:

1 LD R1,0(R2)
2 MUL R7,R8,R9
3 ADD R3,R1,R4
4 SUB R5,R3,R6
5 ADD R10,R1,R11

Exercise 2

Q9. Consider the following instruction sequence:

1. LD R1, 0(R2)

2. ADD R3, R1, R4

3. SUB R5, R3, R6

4. MUL R7, R8, R9

5. ADD R10, R1, R11

Assume: Load latency = 2 cycles; ADD/SUB latency = 1 cycle; MUL latency = 3 cycles; In-order issue pipeline.

Answer the following questions:

a) Identify data dependencies

b) Perform static scheduling to minimize stalls

c) Draw the execution timeline

Execution timeline

Cycle	Instruction
1	LD
2	MUL
3	ADD
4	SUB
5	ADD

Stall removed using instruction reordering.

Exercise 2

Q10. Given the loop:

Loop:

LD F0, 0(R1)

ADD F4, F0, F2

SD F4, 0(R1)

ADDI R1, R1, #8

BNE R1, R3, Loop

Assume: Load latency = 2 cycles; ADD latency = 2 cycles; Store latency = 1 cycle.

Answer the following questions:

a) Unroll the loop 2 times.

b) Apply static scheduling to reduce stalls.

Solutions:

In original loop, dependencies in one iteration:

LD → ADD → SD

ADD needs F0 from LD; SD needs F4 from ADD

So the pipeline has two RAW hazards:

LD → ADD; ADD → SD

Pipeline execution without optimization, timeline example:

Cycle	Instruction
1	LD F0
2	STALL
3	ADD F4
4	STALL
5	SD F4

Two stall cycles appear because: ADD must wait for the loaded value; Store must wait for ADD result.

Exercise 2

Q10. Given the loop:

Loop:

LD F0, 0(R1)

ADD F4, F0, F2

SD F4, 0(R1)

ADDI R1, R1, #8

BNE R1, R3, Loop

Assume: Load latency = 2 cycles; ADD latency = 2 cycles; Store latency = 1 cycle.

Answer the following questions:

a) Unroll the loop 2 times.

b) Apply static scheduling to reduce stalls.

Unroll the loop

```
LD   F0, 0(R1)
ADD  F4, F0, F2
SD   F4, 0(R1)

LD   F6, 8(R1)
ADD  F8, F6, F2
SD   F8, 8(R1)

ADDI R1, R1, #16
BNE  R1, R3, Loop
```

Now we have more independent instructions.

Reorder instructions

```
LD   F0, 0(R1)
LD   F6, 8(R1)
ADD  F4, F0, F2
ADD  F8, F6, F2
SD   F4, 0(R1)
SD   F8, 8(R1)
ADDI R1, R1, #16
BNE  R1, R3, Loop
```

Exercise 2

Q10. Given the loop:

Loop:

LD F0, 0(R1)

ADD F4, F0, F2

SD F4, 0(R1)

ADDI R1, R1, #8

BNE R1, R3, Loop

Assume: Load latency = 2 cycles; ADD latency = 2 cycles; Store latency = 1 cycle.

Answer the following questions:

a) Unroll the loop 2 times.

b) Apply static scheduling to reduce stalls.

Key idea:
The second load fills the stall of the first load.
Now loads hide arithmetic latency.
Pipeline Timeline After Optimization:

Cycle	Instruction
1	LD F0
2	LD F6
3	ADD F4
4	ADD F8
5	SD F4
6	SD F8

Now: ADD F4 executes after enough time for LD F0; ADD F8 executes after LD F6; No pipeline bubbles are needed.
We have the benefits of fewer stalls and more instruction-level parallelism (ILP).

Exercise 2

Q11. Explain why the following pair cannot issue together:

1 ADD R1,R2,R3

2 SUB R4,R1,R5

Solutions:
Instruction 2 uses R1 produced by instruction 1. This is a RAW hazard.
Superscalar processors must perform dependency checking before issuing multiple instructions in the same cycle. Therefore:
Cycle 1: ADD
Cycle 2: SUB
They cannot issue simultaneously.

Exercise 2

Q12. Explain how superscalar processors exploit instruction-level parallelism (ILP) and discuss two major challenges.

Solutions:
Superscalar processors issue multiple instructions per clock cycle **using multiple functional units**.
Challenges:
Dependency checking: RAW, WAR, WAW hazards;
Branch prediction: Branches limit parallel instruction fetch;
Instruction window complexity: Hardware must search many instructions for parallelism.

Closing Remarks and Announcement

Module Overview and Updates

- **Next Week (Week 9)**:

Lecture: In Week 9, our discussion will be focused on some principles of cache memory, and block replacement & write strategy in Lecture 15 and Lecture 16, respectively.

Lab: Instruction Reordering. Instruction reordering can be used for a number of reasons, to **reduce hazards**, including structural hazards, and data hazards. In this exercise we focus on using reordering mainly as a way to improve performance where register hazards persist. For example, we know from previous exercises that RAW hazards cannot be eliminated by renaming, although it can eliminate WAR and WAW hazards. Reordering the instruction sequence can sometimes help to improve if not eliminate these problems. This session will investigate reordering and look at ideas for solving this problem.

Thanks

See you next week!